# Implementing PIKE-RAG on the Collider Architecture: Adapting Knowledge Atomizing and Task Decomposition for Local Constraints

## 1. Executive Summary and Theoretical Foundation

### 1.1 Introduction: The Intersection of Rationale and Resource Constraints

The evolution of Retrieval-Augmented Generation (RAG) systems has reached a critical inflection point. While early implementations ("Naive RAG") focused primarily on semantic similarity search, the current frontier—exemplified by frameworks like PIKE-RAG (Specialized Knowledge and Rationale Augmented Generation)—prioritizes the structural integrity of knowledge and the coherence of reasoning trajectories.[1] This shift is driven by the necessity to handle industrial-grade tasks where simple information retrieval is insufficient, and complex, multi-hop reasoning is required to synthesize accurate answers from specialized corpora.[2]

However, the deployment of such sophisticated reasoning engines has traditionally been the domain of massive cloud clusters utilizing high-bandwidth memory (HBM) GPUs like the NVIDIA A100 or H100. This report investigates a divergent architectural paradigm: the "Collider" architecture. The Collider framework is conceptually defined here as a system that forces the probabilistic outputs of Large Language Models (LLMs) to "collide" with rigid, validated data structures (Pydantic models) within a graph-based execution environment.

The primary research objective is to adapt the core algorithms of PIKE-RAG—specifically **Knowledge Atomizing** and **Knowledge-Aware Task Decomposition**—for operation within the Collider architecture on consumer-grade hardware. The specific constraint of a single GPU with 12GB of VRAM (e.g., NVIDIA RTX 3060) necessitates a radical rethinking of memory management, execution scheduling, and data representation. This report argues that by leveraging **quantization**, **unified virtual memory (UVM)** via RAPIDS cuGraph, and **dynamic Pydantic schema generation**, it is possible to replicate the high-fidelity reasoning of PIKE-RAG in a localized, privacy-centric environment.

### 1.2 The PIKE-RAG Paradigm: Beyond Chunking

Standard RAG methodologies operate on the assumption that a document chunk, retrieved via vector similarity, contains the necessary answer. PIKE-RAG challenges this by positing that "chunks" are arbitrary segmentation artifacts that often sever logical connections or bury key facts in noise. Instead, PIKE-RAG introduces the concept of **Knowledge Atomizing**, which

decomposes complex documents into indivisible units of information—atomic facts—often represented as question-answer pairs or structured triples.[2]

This atomization is coupled with **Knowledge-Aware Task Decomposition**. Unlike standard Chain-of-Thought (CoT) prompting, which hallucinates a plan based on linguistic patterns, Knowledge-Aware decomposition creates a plan based on *available* knowledge. It effectively "looks before it leaps," verifying that the necessary atomic facts exist in the index before committing to a reasoning path.[4] The analysis suggests that this approach is particularly potent for "Linkable-Reasoning" questions—those requiring the synthesis of disparate facts—where PIKE-RAG has demonstrated up to 20% improvement over baselines on benchmarks like MuSiQue.[5]

### 1.3 The Collider Architecture: Structural collision

For the purposes of this implementation analysis, the **Collider** architecture is formalized as a Graph-of-Thought (GoT) engine where every node transition is governed by a Pydantic contract. In a resource-constrained environment, "lazy" or unstructured data is a liability. By enforcing strict schemas at every step (Input/Output), the Collider minimizes the context window bloat associated with correcting malformed LLM outputs and allows for efficient offloading of state to CPU RAM.

The architecture relies on three distinct computational substrates:

1. **The Probabilistic Substrate:** A quantized Local LLM (e.g., Llama-3-8B-Instruct-GGUF) responsible for semantic generation and atom extraction.[6]
2. **The Deterministic Substrate:** A `pydantic-graph` state machine that orchestrates the workflow, ensuring that reasoning steps follow a logical directed acyclic graph (DAG).[7]
3. **The Structural Substrate:** A local graph database powered by RAPIDS cuGraph (via NetworkX backends) to perform structural analysis on the atomized knowledge.[9]

The synthesis of these elements creates a system capable of "colliding" the fluidity of natural language with the rigidity of logic programming, all within the thermal and memory envelope of a consumer workstation.

---

## 2. Adapting Knowledge Atomizing: From Text to Structured Atoms

### 2.1 The Theoretical Imperative of Atomization

The foundational premise of Knowledge Atomizing is that the optimal unit of retrieval is not the unit of storage (the document chunk) but the unit of meaning (the atomic fact). In industrial applications, a single paragraph might contain specifications for voltage, thermal limits, and physical dimensions. A vector search for "thermal limits" might retrieve this paragraph, but a subsequent reasoning step requiring "voltage" might miss it if the embedding was dominated by thermal terminology.[2]

PIKE-RAG solves this through a "Dual Rewriting" strategy.[3] The system rewrites the raw text into a set of "Atomic Questions"—questions that the text definitively answers. This aligns the retrieval index with the user's query space. If a user asks about "voltage," the system matches against the generated question "What is the voltage rating?" rather than the raw text.

## 2.2 Algorithm 1: The Collider Atomization Pipeline

Adapting this for the Collider architecture requires formalizing the "Atomic Unit" as a Pydantic model. This ensures type safety and facilitates downstream graph construction.

**Pipeline Definition:**

1. **Ingestion & Segmentation:**
   - Documents are ingested and split using a semantic chunker (e.g., `RecursiveCharacterTextSplitter`).
   - *Constraint Check:* On 12GB VRAM, batch processing must be strictly serialized. We cannot process multiple documents in parallel on the GPU.[10]
2. **Atom Extraction (The "Atomizer"):**
   - The core transformation occurs here. The LLM is prompted to extract facts.
   - **Prompt Engineering:** The prompt must be designed to output structured JSON matching the Pydantic schema.
   - *Template:* "You are a Knowledge Engineer. Extract all independent facts from the text. For each fact, generate a specific 'Atomic Question' that the fact answers, and the 'Atomic Answer' derived solely from the text.".[11]
3. **Schema Validation (The Collision):**
   - The LLM output is parsed by Pydantic.
   - If validation fails (e.g., missing fields, hallucinated types), the Collider rejects the atom and triggers a retry or logs the error. This "collision" with the schema filters out noise and low-quality generations.
4. **Indexing:**
   - **Vector Index:** The `atomic_question` is embedded using a lightweight model (e.g., `all-MiniLM-L6-v2` or `nomic-embed-text`) running on the CPU or GPU.[10]
   - **Graph Index:** Entities within the `atomic_answer` are extracted (via Named Entity Recognition) and inserted into the `cuGraph` structure as nodes, with edges linking them to the source document and related atoms.

### 2.3 Data Structure Design: The Pydantic Substrate

To enable efficient local execution, the data structures must be minimized. We utilize pydantic.create_model for dynamic adaptability, allowing the system to adjust the schema based on the domain (e.g., adding a "citation" field for legal documents).[13]

**Table 1: The AtomicKnowledgeUnit Schema Specification**

| Field Name | Data Type | Validation Logic | System Role |
|---|---|---|---|
| atom_id | UUID | default_factory=uuid4 | Primary Key for graph linkage. |
| source_ref | str | min_length=1 | Hash of the source chunk (traceability). |
| atomic_q | str | min_length=10, max_length=150 | The embedding target. Represents the "searchable surface" of the fact. |
| atomic_a | str | min_length=1 | The factual payload. Used for synthesis. |
| entities | List[Entity] | default_factory=list | Extracted entities for Knowledge Graph construction. |
| confidence | float | ge=0.0, le=1.0 | Model's self-assessed confidence (filtering threshold). |

Implication of this Structure:

By forcing knowledge into this rigid structure, we decouple the storage of information from the representation of information. The atomic_q serves as a semantic hook, while the atomic_a serves as the payload. This separation is crucial for the "Dual Rewriting" mechanism described in PIKE-RAG.15

### 2.4 Prompt Engineering for Atomization

The effectiveness of the Atomizer depends entirely on the prompt's ability to elicit granular decompositions. The snippet [11] provides a template for the decomposition phase, which we adapt here for the atomization phase.

**Prompt Strategy:**

> "Analyze the provided text segment. Break it down into the smallest possible units of independent meaning (Atomic Facts). For each fact, formulate a question that can be answered *unambiguously* by that fact alone. Return the result as a list of JSON objects conforming to the following schema: {question: str, answer: str, entities: list[str]}."

This approach mimics the "Knowledge Atomizing" process by forcing the model to dual-code the information: once as a declarative fact (Answer) and once as an interrogative index (Question).[3]

---

## 3. Knowledge-Aware Task Decomposition: The Dependency Graph Engine

### 3.1 The Concept of Knowledge-Awareness

Traditional "Decompose-then-Solve" methods (like Least-to-Most Prompting) suffer from a critical flaw: they decompose tasks based on linguistic intuition rather than knowledge availability. An LLM might decompose a question about "RTX 5090 specs" into "Find CUDA core count" and "Find VRAM size," unaware that the RTX 5090 has not been released and no such documents exist in the corpus. This leads to hallucination chains.

PIKE-RAG introduces **Knowledge-Aware Decomposition**. The decomposer proposes sub-questions, but before adding them to the execution plan, the system *verifies* their feasibility against the index.[4] In the Collider architecture, this verification step is the "Knowledge-Aware Check."

### 3.2 Algorithm 2: The Recursive Dependency Graph

We model this process as a dynamic state machine using **pydantic-graph** (currently in beta). This library allows us to define nodes as functions that return the next state, enabling cyclic, recursive, and branching logic.[7]

**The Execution Loop:**

1. **State Initialization:**
   - The graph is initialized with a `ReasoningState` object containing the `original_query` and an empty `knowledge_context`.
2. **Node 1: The Proposer (LLM):**
   - Input: `ReasoningState`.
   - Task: Generate a list of candidate `atomic_sub_questions`.
   - *Constraint:* "Generate questions that *might* be answered by the corpus. Do not attempt to answer them yet."
3. **Node 2: The Verifier (Vector Search):**
   - Input: Candidate `atomic_sub_questions`.
   - Action: For each candidate, query the `AtomicKnowledgeUnit` vector index.
   - *Thresholding:* If the top retrieval score is below a threshold (e.g., 0.75), the sub-question is marked as "Unanswerable/Missing Data."
   - *Result:* A list of `ValidatedSubTask` objects. This is the **Knowledge-Aware** filter. It prevents the system from pursuing dead ends.
4. **Node 3: The Executioner (Retrieval & Synthesis):**
   - For valid sub-tasks, retrieve the `atomic_a` (Answer).
   - Update `knowledge_context`.
5. **Node 4: The Router (Decision Logic):**
   - Input: `knowledge_context` + `original_query`.
   - Action: Determine if sufficient information exists to answer the user.
   - *Branch:*
     - If YES -> Transition to **FinalSynthesisNode**.
     - If NO -> Transition back to **ProposerNode** (Recursion) to generate deeper questions based on new context.

## 3.3 Implementing Dependency Graphs with Pydantic-Graph

The use of `pydantic-graph` allows this entire logic to be type-checked and serialized. The state object becomes the "memory" of the agent.

**Code Structure Concept:**

```python
from pydantic_graph import BaseNode, Graph, End
from pydantic import BaseModel
```

```
class ReasoningState(BaseModel):
    query: str
    context: List[AtomicKnowledgeUnit]
    depth: int

class ProposerNode(BaseNode):
    async def run(self, ctx):
        # LLM Logic to propose sub-questions
        #...
        return VerifierNode()

class VerifierNode(BaseNode):
    async def run(self, ctx):
        # Vector Search Logic (Knowledge-Awareness)
        # Filter out low-confidence queries
        #...
        if sufficient_info:
            return End(Result(...))
        else:
            return ProposerNode() # Loop
```

This graph-based approach ensures that the reasoning process is explicit, traceable, and capable of self-correction—key requirements for the "Rationale Augmented" aspect of PIKE-RAG.[1]

---

# 4. Hardware Optimization: The 12GB VRAM Frontier

The implementation of PIKE-RAG on an RTX 3060 (12GB VRAM) defines the physical boundaries of the Collider architecture. 12GB is the "uncanny valley" of AI hardware: enough to run competent models, but insufficient to run them comfortably alongside heavy auxiliary systems (Graph DBs, Vector Stores) without careful orchestration.

## 4.1 The VRAM Budget: A Zero-Sum Game

To run PIKE-RAG locally, we must partition the 12GB resource. The simultaneous execution of an LLM, a Graph Neural Network (GNN), and a dense vector index is impossible. Therefore, the Collider architecture mandates a **Sequential Execution Pipeline**.[17]

**Resource Allocation Breakdown:**

| Component | Technology Strategy | VRAM Usage (Est.) | System RAM Usage |
|---|---|---|---|
| **System Overhead** | Minimal Desktop / Headless Linux | 0.5 - 1.5 GB | - |
| **LLM (Reasoning)** | **Llama-3-8B (Q4_K_M GGUF)** via llama.cpp | ~5.5 GB | - |
| **LLM Context (KV)** | 8192 Token Context Window | ~2.0 GB | - |
| **Vector Search** | **CPU-Based HNSW** (Qdrant/FAISS) | **0 GB** (Offloaded) | ~2-4 GB |
| **Graph Analytics** | **RAPIDS cuGraph** (nx-cugraph) | ~2.0 GB (Dynamic) | 16GB+ (Spillover) |
| **Buffer** | Fragmentation / Temporary Tensors | ~1.0 GB | - |

Analysis:

This allocation leaves approximately 1GB of headroom. This is critical. If the LLM context grows too large (e.g., deep conversation history), it will trigger OOM. Therefore, Knowledge Atomizing is not just a reasoning improvement; it is a hardware necessity. By retrieving only atomic facts (dense information) rather than raw chunks (sparse information), we minimize context usage, keeping the KV cache within the 2GB budget.[19]

## 4.2 Optimizing the LLM Layer

For the 12GB constraint, 4-bit quantization (GGUF or AWQ) is mandatory.[20]

- **GGUF (llama.cpp):** Best for "split" execution. If VRAM runs out, layers automatically offload to CPU RAM. This prevents crashes but kills performance (tokens/sec drops from ~40 to ~2).
- **vLLM vs. Ollama:** vLLM is designed for high-throughput batching and reserves massive blocks of VRAM for the KV cache (PagedAttention). On a single 12GB card, vLLM often causes OOM errors when co-located with other GPU tasks.[22] **Ollama** (or llama.cpp directly) is preferred for the Collider architecture because its memory footprint is more flexible and it releases VRAM more aggressively when idle.[24]

### 4.3 Optimizing the Graph Layer: RAPIDS cuGraph & UVM

PIKE-RAG implies graph-based reasoning (though not explicitly requiring a GNN for all steps). Using **RAPIDS cuGraph** allows for GPU-accelerated graph algorithms (PageRank, Community Detection) which can uncover latent structures in the retrieved atoms.

- **The Problem:** Storing a graph on GPU takes memory.
- **The Solution: Unified Virtual Memory (UVM)**. Modern CUDA drivers allow the GPU to access system RAM transparently. If the graph grows beyond the allocated 2GB VRAM partition, cuGraph spills data to the system RAM over the PCIe bus.[26] This causes a performance penalty (PCIe bandwidth vs VRAM bandwidth), but it enables the processing of graphs (e.g., 100k+ nodes) that would otherwise crash the system.
- **nx-cugraph:** We utilize the NetworkX backend (nx-cugraph), which allows us to write standard, Pythonic NetworkX code. The backend automatically dispatches operations to cuGraph if the GPU is available and data fits, fallbacking or spilling as necessary. This enables "Zero Code Change" acceleration.[9]

### 4.4 Optimizing the Vector Layer: The CPU Fallback

While GPU-accelerated vector search (e.g., **RAPIDS cuVS**) offers incredible speed, it requires the index to be loaded into VRAM. For a 12GB card running an 8B LLM, this is a luxury we cannot afford.

- **Decision:** The Collider architecture explicitly designates Vector Search as a **CPU Task**. Modern CPU implementations of HNSW (in Qdrant, Chroma, or FAISS) can search 1M vectors in milliseconds.[28] The latency trade-off is imperceptible in a single-user agentic workflow, but the VRAM savings (0.5-1GB) are critical for LLM stability.

---

## 5. Integrating Pydantic: The Dynamic Schema Engine

The Collider architecture relies on Pydantic not just for validation, but for **dynamic tool creation**. In a specialized RAG system, the "tools" available to the agent might change based on the document type.

### 5.1 Runtime Model Creation

Using `pydantic.create_model`[13], the system can generate schemas on the fly.

- *Scenario:* The user uploads a CSV of financial data.
- *Action:* The system analyzes the CSV headers and generates a Pydantic model `FinancialRecord` with typed fields (`revenue: float`, `date: datetime`).
- *Integration:* This dynamic model is then registered as a tool for the LLM using `pydantic-ai` decorators.[29] This allows the LLM to interact with the new data structure immediately, enforcing type safety on its queries.

### 5.2 Tool Registration via Pydantic-AI

The `pydantic-ai` library facilitates the binding of these models to the agent.

- **Function Tools:** We define python functions (e.g., `graph_traversal`) and annotate their arguments with Pydantic models. The framework automatically converts these into the JSON schema required by the LLM's function calling API.[31]
- **Dynamic Registration:** Tools can be added to the agent context at runtime. If the "Decomposition Node" determines that a specific mathematical analysis is needed, it can register a `math_tool` for the subsequent "Solver Node," keeping the context window clean of irrelevant tool definitions.[29]

---

## 6. Synthesis: The Collider Execution Flow

Combining all elements, we define the full execution lifecycle of a PIKE-RAG query on the Collider architecture.

### 6.1 Phase 1: Ingestion (The Atomizer)

- *Hardware State:* GPU is dedicated to LLM (Quantized). Vector DB is on CPU.
- *Process:* Document -> Chunk -> LLM (Extract Atoms via Prompt) -> Validate (Pydantic) -> Index (Qdrant CPU).
- *Result:* A dense index of `AtomicQuestions` and a Graph of `Entities`.

### 6.2 Phase 2: Reasoning (The Graph Runner)

- *Hardware State:* GPU hosts LLM. System RAM hosts `pydantic-graph` state.

- *Process:*
    1. User Query -> **Decompose Node** (LLM).
    2. Generated Sub-Questions -> **Verify Node** (CPU Vector Search).
        - *Knowledge-Aware Check:* Only keep questions with high similarity scores in the index.
    3. Validated Questions -> **Retrieve Node** (CPU Vector Search + Optional cuGraph Traversal).
        - *cuGraph:* If structural query (e.g., "most influential paper"), load subgraph to GPU (swapping LLM layers if needed) -> Calculate -> Return to RAM.
    4. Aggregated Atoms -> **Synthesize Node** (LLM).
    5. Final Output.

### 6.3 Phase 3: Rationale Construction

The `ReasoningState` object in `pydantic-graph` accumulates the trace of this traversal. When the final answer is generated, the system can output the "Rationale" by printing the visited nodes and the atoms retrieved at each step. This fulfills PIKE-RAG's requirement for "constructing coherent rationale".[4]

---

## 7. Implications and Strategic Insights

### 7.1 The Density-Latency Trade-off

Implementing PIKE-RAG locally reveals a crucial second-order insight: Knowledge Atomizing acts as a compression algorithm for reasoning.

Standard RAG retrieves "context chunks" that are often 80% noise. This noise consumes the scarce KV cache (VRAM) and dilutes the LLM's attention. By atomizing knowledge, we increase the "information density" of the retrieved context. A 12GB GPU can effectively "know" more because it isn't wasting VRAM on irrelevant tokens. The trade-off is increased ingestion latency (due to the atomization processing), but the runtime inference becomes faster and more accurate.19

### 7.2 The Shift to "White-Box" Reasoning

The Collider architecture, through `pydantic-graph`, moves AI reasoning from a "Black Box" (one giant prompt) to a "White Box" (traceable graph state). In industrial settings, this is invaluable. If the system fails to answer, an engineer can inspect the graph state to see exactly which sub-question failed the "Knowledge-Aware" verification step. This traceability is a direct consequence of enforcing Pydantic schemas on the reasoning steps.

### 7.3 The Viability of Consumer Hardware

This research confirms that the 12GB VRAM limit is not a hard barrier for advanced RAG, provided one abandons the "all-in-GPU" mindset. By adopting a **heterogeneous compute model**—CPU for Vectors, GPU for Reasoning, UVM for Graphs—and enforcing strict **data serialization** via Pydantic, consumer hardware can emulate the architectural sophistication of enterprise systems. The Collider architecture demonstrates that *structure* can substitute for *scale*.

---

# 8. Conclusion

The adaptation of PIKE-RAG into the Collider architecture creates a robust framework for high-fidelity, local reasoning. By rigorously implementing **Knowledge Atomizing** via Pydantic validation, the system ensures that the limited context window of a local LLM is populated only with high-value, dense information. The **Knowledge-Aware Task Decomposition**, realized through a pydantic-graph state machine, prevents the hallucination loops common in smaller models by grounding every reasoning step in verifiable data availability.

While the 12GB VRAM constraint of the RTX 3060 imposes strict limits—necessitating quantization, sequential execution, and CPU offloading—these limitations effectively enforce a disciplined architectural design that prioritizes efficiency and precision. The resulting system is not merely a "scaled-down" version of a cloud RAG; it is a privacy-preserving, transparent, and highly specialized reasoning engine capable of tackling complex industrial queries without leaving the local network.

---

# 9. References and Data Sources

7
.